PROGRAMAÇÃO FUNCIONAL

ANÁLISE SINTÁTICA (PARSING)

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

O QUE É PARSING?

No **mundo real**, dados chegam aos sistemas em formatos de **texto plano** — arquivos, requisições **HTTP**, entradas de **terminal** ou **scripts**.

Para o computador, uma **String** é apenas uma **sequência unidimensional** e **amorfa** de **caracteres** (ex: "18.5" ou "[1, 2, 3]").

O QUE É PARSING?

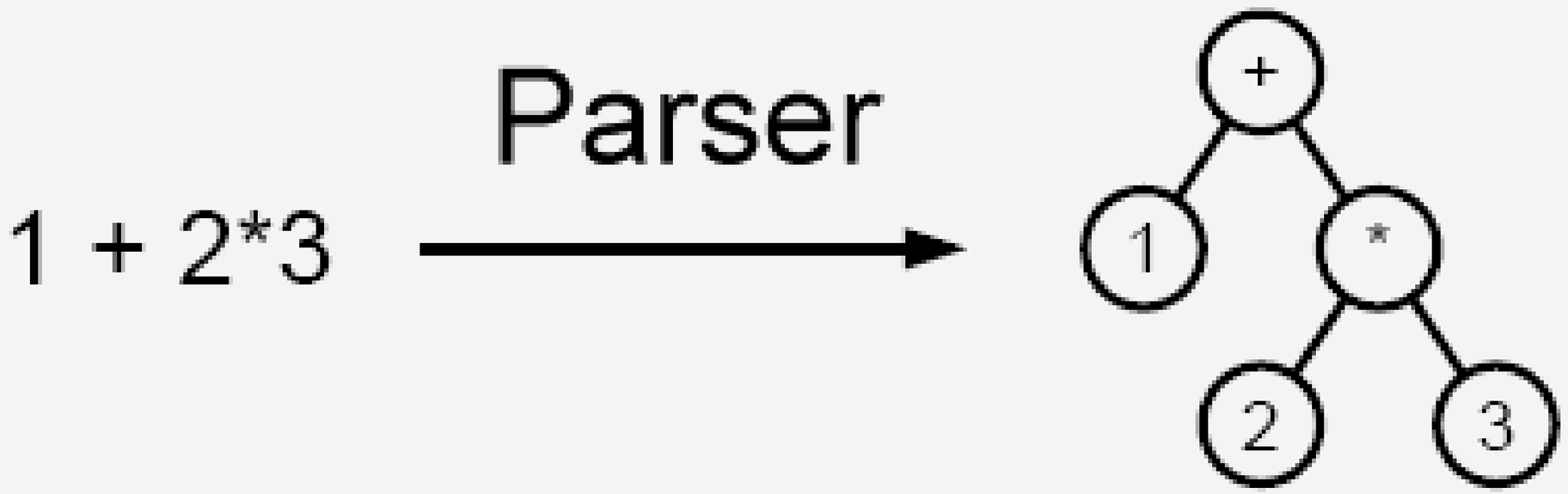
Para aplicar **lógica de negócios**, validar **regras** e realizar **cálculos**, precisamos **converter** esse **texto** sem estrutura em **tipos algébricos** de dados (Int, Double, estruturas customizadas ou Árvores de Sintaxe Abstrata - AST).

O QUE É PARSING?

Definição de Parsing (Análise Sintática):
É o processo de **analisar** uma **sequência** de **símbolos** de entrada (texto) e construir uma **representação estruturada** em memória que obedece a uma **gramática bem definida**.

O QUE É PARSING?

O Fluxo Fundamental de Processamento:



O QUE É PARSING?

O Problema da Conversão Ad-Hoc(Litoral): Escrever **manipuladores** de **string** manuais com **laços** e manipulação de **ponteiros**/**índices** é propenso a **erros**, difícil de **manter** e **inseguro** quanto a falhas de sintaxe.

LIMITAÇÕES DA ABORDAGEM AD-HOC

Fragilidade quanto a Falhas: Manipulações manuais de string frequentemente lançam exceções em tempo de execução (runtime errors) ao receberem entradas ligeiramente fora do formato esperado.

LIMITAÇÕES DA ABORDAGEM AD-HOC

Incapacidade de Tratar Estruturas Aninhadas/Recursivas:
Expressões com parênteses balanceados ou operadores encadeados (ex: add 18 (add 5 2)) não podem ser analisadas de forma confiável apenas com divisões simples de string ou REGEX.

LIMITAÇÕES DA ABORDAGEM AD-HOC

Acoplamento Severo:
A lógica de leitura do texto fica misturada com a lógica de execução, tornando a manutenção extremamente complexa.

SINTAXE CONCRETA VS. SEMÂNTICA

Sintaxe Concreta:
Representa a **sequência exata** de caracteres **digitada** pelo **usuário** no arquivo ou terminal. É **dependente** da **formatação visual** escolhida.

SINTAXE CONCRETA VS. SEMÂNTICA

Semântica: Representa o **significado**, a **regra de negócio** ou o **valor matemático** resultante da computação.

SINTAXE CONCRETA VS. SEMÂNTICA

Árvore de Sintaxe Abstrata (AST):
Atua como uma **ponte** neutra de dados (um Tipo Algébrico de Dados em Haskell) que **desvincula** a **sintaxe concreta** do seu significado **semântico**.

SINTAXE CONCRETA VS. SEMÂNTICA

Observe como **três sintaxes concretas** completamente diferentes possuem a **mesma representação abstrata** (AST) e produzem a **mesma semântica**:

- Sintaxe Concreta 1 (Infixa): "18 + 5"
- Sintaxe Concreta 2 (Prefixa): "add 18 5"
- Sintaxe Concreta 3 (Estilo Lisp): "(+ 18 5)"

Add (Literal 18) (Literal 5)

ESPECIFICAÇÃO FORMAL DE LINGUAGENS

Para construir um **parser correto**, devemos primeiro definir com precisão matemática as **regras** do **texto** que aceitaremos.
Utilizamos para isso uma Gramática Livre de Contexto escrita na notação BNF (Backus-Naur Form)**.

ESPECIFICAÇÃO FORMAL DE LINGUAGENS

<Expr> ::= <Numero>
        | "add " <Expr> " " <Expr>
        | "mul " <Expr> " " <Expr>

<Numero> ::= <Digito>
        | <Digito> <Numero>

<Digito> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'

ESPECIFICAÇÃO FORMAL DE LINGUAGENS

```
-- Tipo Algébrico de Dados que reflete a gramática formal
data Expr = Literal Int
          | Add Expr Expr
          | Mul Expr Expr
          deriving (Show, Eq)
```

PROBLEMAS COMPUTACIONAIS E APLICAÇÕES PRÁTICAS

Processamento de Arquivos Estruturados:

- CSV (Comma-Separated Values): Leitura e conversão de tabelas textuais em listas de registros tipados.
- Formatos de Intercâmbio: Parsers para JSON, XML e YAML em APIs Web.

PROBLEMAS COMPUTACIONAIS E APLICAÇÕES PRÁTICAS

Interpretadores e Compiladores:

- Tradução de código-fonte de linguagens de programação (C, Haskell, Python) para representações intermediárias.
- Processamento de linguagens de consulta como SQL.

PROBLEMAS COMPUTACIONAIS E APLICAÇÕES PRÁTICAS

Linguagens de Domínio Específico (DSLs):

- Avaliação de expressões matemáticas, regras de filtragem ou arquivos de configuração customizados.

Validação e Sanitização de Entradas:

- Verificação de conformidade sintática para formatos como e-mails, URLs, datas e documentos (CPF/CNPJ).

DEFININDO UM TIPO DE PARSER SIMPLES

Como representar um "**analisador de texto**" de forma pura em Haskell?

Um **parser** precisa **consumir** o início de uma **String**, extrair o **dado** desejado e devolver o **restante** do **texto** que sobrou para as etapas seguintes.

DEFININDO UM TIPO DE PARSER SIMPLES

Definimos o tipo **Parser a** encapsulando uma função que recebe uma **String** e retorna **Maybe (a, String)**:

```
newtype Parser a = Parser { runParser :: String
-> Maybe (a, String) }
```

DEFININDO UM TIPO DE PARSER SIMPLES

Just (dado, resto): O texto inicial casava com o padrão esperado.

Nothing: O texto não respeitava a sintaxe esperada (falha no parse).

CONVERTENDO STRINGS DE ENTRADA EM COMANDOS DE SISTEMA

Definição de duas funções auxiliares puras para tratamento de espaços e separação de palavras:

CONVERTENDO STRINGS DE ENTRADA EM COMANDOS DE SISTEMA

```
-- Remove espacos em branco do inicio do
texto
removerEspacos :: String -> String
removerEspacos [] = []
removerEspacos (c:cs)
    | c == ' '  = removerEspacos cs
    | otherwise = c:cs
```

CONVERTENDO STRINGS DE ENTRADA EM COMANDOS DE SISTEMA

```
-- Separa a primeira palavra do restante do texto
extrairPalavra :: String -> (String, String)
extrairPalavra texto = extrair (removerEspacos texto)
  where
    extrair [] = ("", "")
    extrair (c:cs)
        | c == ' '  = ("", c:cs)
        | otherwise =
            let (palavra, resto) = extrair cs
            in (c : palavra, resto)
```

PARSER DE COMANDOS DE USUÁRIO

```
-- Tipo Algebrico de Dados representando os comandos do sistema
data Comando = Login String
             | Logout
             deriving (Show, Eq)
```

PARSER DE COMANDOS DE USUÁRIO

```
-- Parser que converte a String de entrada no Comando
parseComando :: Parser Comando
parseComando = Parser $ \input ->
    let (tag, resto1) = extrairPalavra input
    in case tag of
        "LOGIN" ->
            let (usuario, resto2) = extrairPalavra resto1
            in if null usuario
               then Nothing
               else Just (Login usuario, resto2)
        "LOGOUT" -> Just (Logout, resto1)
        _        -> Nothing
```

SIMULAÇÃO NO REPL (GHCI):

```
> runParser parseComando "LOGIN marcos"
Just (Login "marcos", "")

> runParser parseComando "LOGOUT
sessao123"
Just (Logout, "sessao123")

> runParser parseComando "ENTRAR marcos"
Nothing
```

DÚVIDAS???